

/**/

PRÁCTICA CALIFICADA N° 02 – SEMANA 3

I. OBJETIVOS

- Aplicar el uso de objetos constantes y funciones constantes.
 - Implementar composición de clases (objetos como miembros).
 - Utilizar funciones friend y el puntero this en C++.
 - Comparar los conceptos de encapsulamiento y visibilidad entre C++ y Python.
 - Analizar cómo se implementan conceptos equivalentes en Python.
-

II. CONTENIDO DE LA PRÁCTICA

Se trabajará sobre los siguientes conceptos clave:

- Objetos y funciones const.
 - Composición (clases que contienen otras clases).
 - Funciones amigas (friend).
 - El puntero this y llamadas encadenadas (return *this).
 - Creación dinámica de objetos (new y delete).
 - Comparativa estructurada entre C++ y Python.
-

III. ACTIVIDADES

Actividad 3.1 – Constantes y Funciones Constantes

Enunciado:

Utilizando la clase Time vista en la práctica anterior, declare un objeto constante const Time, e intente utilizar métodos set y get para analizar qué funciones requieren ser const.

Instrucciones:

- En C++: Implementa getHour () const, printMilitary () const, etc.
- En Python: Simula el concepto de inmutabilidad usando propiedades @property.

Comparar comportamiento.

[Repositorio de GitHub](#)

https://github.com/aymar2025v/POO_II_OFI

ruta en línea de los archivos

<https://www.onlinegdb.com/edit/BhjHKGyQJ>

Time.h

```
#ifndef TIME_H
#define TIME_H

#include <iostream>

class Time {
public:
    Time(int h = 0, int m = 0, int s = 0);

    // Métodos set (modificadores) - NO son const
    void setTime(int h, int m, int s);
    void setHour(int h);
    void setMinute(int m);
    void setSecond(int s);
```

```
/**/
```

```
// Métodos get (consultores) - Sí son const
int getHour() const;
int getMinute() const;
int getSecond() const;

// Métodos de impresión (no modifican) - Sí const
void printMilitary() const;
void printStandard() const;

private:
    int hour;
    int minute;
    int second;
};

#endif
```

<https://www.onlinegdb.com/edit/BhjHKGyQJ#>

Time.cpp

```
#include "Time.h"
#include <iostream>
#include <iomanip>
using namespace std;

Time::Time(int h, int m, int s) : hour(h), minute(m), second(s) {}

void Time::setTime(int h, int m, int s) {
    setHour(h);
    setMinute(m);
    setSecond(s);
}

void Time::setHour(int h) {
    hour = (h >= 0 && h < 24) ? h : 0;
}

void Time::setMinute(int m) {
    minute = (m >= 0 && m < 60) ? m : 0;
}

void Time::setSecond(int s) {
    second = (s >= 0 && s < 60) ? s : 0;
}

// Métodos const
int Time::getHour() const {
    return hour;
}

int Time::getMinute() const {
    return minute;
}
```

/**/

```
int Time::getSecond() const {
    return second;
}

void Time::printMilitary() const {
    cout << setfill('0') << setw(2) << hour << ":"
        << setw(2) << minute << ":" << setw(2) << second;
}

void Time::printStandard() const {
    int h = (hour == 0 || hour == 12) ? 12 : hour % 12;
    cout << h << ":" << setfill('0') << setw(2) << minute << ":"
        << setw(2) << second << (hour < 12 ? " AM" : " PM");
}
```

<https://www.onlinegdb.com/edit/BhjHKGyQJ#>

main.cpp

```
#include "Time.h"
#include <iostream>
using namespace std;

int main() {
    // --- Objeto no constante ---
    Time t1(10, 30, 45);
    cout << "t1 (no const): ";
    t1.printMilitary();
    cout << endl;

    t1.setHour(14);
    t1.printMilitary();
    cout << endl;

    // --- Objeto constante ---
    const Time t2(23, 59, 58);
    cout << "\nt2 hora (getHour): " << t2.getHour() << endl;
    cout << "t2 (const): ";
    t2.printMilitary();
    cout << endl;

    // Esto NO compila: setHour no es const
    // t2.setHour(22); // ERROR: passing 'const Time' as 'this'
    argument discards qualifiers

    // Esto NO compila: setMinute no es const
    // t2.setMinute(0); // ERROR similar

    // Podemos leer todos los getters
    cout << "t2 minutos: " << t2.getMinute() << endl;
    cout << "t2 segundos: " << t2.getSecond() << endl;

    return 0;
}
```

<https://www.onlinegdb.com/edit/BhjHKGyQJ#>

/**/

Salida esperada

```
t1 (no const): 10:30:45
14:30:45

t2 hora (getHour): 23
t2 (const): 23:59:58
t2 minutos: 59
t2 segundos: 58
```

implementación en python

```
class Time:
    def __init__(self, hour=0, minute=0, second=0):
        self._hour = hour
        self._minute = minute
        self._second = second
        self._validate()

    def _validate(self):
        """Asegura que los valores estén en rango."""
        if not (0 <= self._hour < 24):
            self._hour = 0
        if not (0 <= self._minute < 60):
            self._minute = 0
        if not (0 <= self._second < 60):
            self._second = 0

    # Propiedades de solo lectura (getters)
    @property
    def hour(self):
        return self._hour

    @property
    def minute(self):
        return self._minute

    @property
    def second(self):
        return self._second

    # Setters (opcionales, para objetos modificables)
    @hour.setter
    def hour(self, value):
        self._hour = value
        self._validate()

    @minute.setter
    def minute(self, value):
        self._minute = value
        self._validate()

    @second.setter
    def second(self, value):
        self._second = value
        self._validate()
```

/**/

```
def set_time(self, h, m, s):
    self.hour = h
    self.minute = m
    self.second = s

def print_military(self):
    print(f"{self.hour:02d}:{self.minute:02d}:{self.second:02d}")

def print_standard(self):
    h = self.hour % 12
    if h == 0:
        h = 12
    ampm = "AM" if self.hour < 12 else "PM"
    print(f"{h}:{self.minute:02d}:{self.second:02d} {ampm}")

# --- Objeto modificable (normal) ---
t1 = Time(10, 30, 45)
t1.print_military()          # 10:30:45
t1.hour = 14                 # Usa el setter
t1.print_military()          # 14:30:45

# --- Simulación de objeto constante (sin setters) ---
class ConstTime:
    """Versión de solo lectura: no se definen setters."""
    def __init__(self, hour, minute, second):
        self._hour = hour
        self._minute = minute
        self._second = second
        # No hay validación automática; podríamos hacerla en __init__

    @property
    def hour(self):
        return self._hour

    @property
    def minute(self):
        return self._minute

    @property
    def second(self):
        return self._second

    def print_military(self):
        print(f"{self.hour:02d}:{self.minute:02d}:{self.second:02d}")

    def print_standard(self):
        h = self.hour % 12
        if h == 0:
            h = 12
        ampm = "AM" if self.hour < 12 else "PM"
        print(f"{h}:{self.minute:02d}:{self.second:02d} {ampm}")

# --- Prueba del objeto constante simulado ---
t2 = ConstTime(23, 59, 58)
print(f"t2 hour (propiedad): {t2.hour}")    # 23
t2.print_military()                          # 23:59:58
```

/**/

```
# Intentar modificar genera un AttributeError porque no hay setter
try:
    t2.hour = 22
except AttributeError as e:
    print(f"Error (esperado): {e}")    # can't set attribute

# Nota: Aún se podría acceder a t2._hour directamente (convención de
# privacidad),
# pero el lenguaje no lo impide. La simulación es solo por convención.
```

<https://www.onlinegdb.com/edit/l3oMheax->

Salida esperada

```
10:30:45
14:30:45
t2 hour (propiedad): 23
23:59:58
Error (esperado): property 'hour' of 'ConstTime' object has no setter
```

CUADRO COMPARATIVO

Aspecto	C++	Python
Palabra clave	const (se usa en objetos y métodos)	No existe. Se simula con @property sin setter o con namedtuple
Objeto constante	Const Timet(10,30,45); → no se puede modificar	No hay equivalente directo. Se crea una clase sin métodos modificadores
Método de solo lectura	int getHour() const; → el compilador garantiza que no modifica el objeto	@property + método getter → indica intención de solo lectura, pero no lo impone el lenguaje
¿Qué pasa si intento modificar?	Error de compilación (el programa ni siquiera se crea)	Error en tiempo de ejecución (si la propiedad no tiene setter) o simplemente se modifica (si se accede directamente al atributo)
Seguridad	Alta – el compilador protege el objeto constante	Baja – se confía en la disciplina del programador
Ejemplo	t.getHour() / t.setHour(5) (no compila)	t.hour / t.hour = 5 (AttributeError si no hay setter)

/**/

Actividad 3.2 – Composición de Clases

Enunciado:

Crea una clase Empleado que contenga dos objetos tipo Fecha:

- FechaNacimiento
- FechaContratacion

Instrucciones:

- Implementa ambas clases (Empleado y Fecha) en C++ y Python.
- Asegúrate de inicializar los objetos miembros correctamente en el constructor.

Comparar el orden de creación y destrucción de objetos.

Fecha.h

```
#ifndef FECHA_H
#define FECHA_H

#include <iostream>

class Fecha {
public:
    // Constructor con valores por defecto
    Fecha(int dia = 1, int mes = 1, int anio = 2000);

    // Destructor (para observar el orden)
    ~Fecha();

    void mostrar() const;

private:
    int dia, mes, anio;
};

#endif
```

<https://www.onlinegdb.com/edit/rtDHk95fxW>

Fecha.cpp

```
#include "Fecha.h"
#include <iostream>

Fecha::Fecha(int d, int m, int a)
    : dia(d), mes(m), anio(a) {
    std::cout << "Constructor de Fecha: " << dia << "/" << mes << "/"
<< anio << std::endl;
}

Fecha::~~Fecha() {
    std::cout << "Destructor de Fecha: " << dia << "/" << mes << "/"
<< anio << std::endl;
}
```

/**/

```
void Fecha::mostrar() const {  
    std::cout << dia << "/" << mes << "/" << anio;  
}
```

<https://www.onlinegdb.com/edit/rtDHk95fxW>

Empleado.h

```
#ifndef EMPLEADO_H  
#define EMPLEADO_H  
  
#include <string>  
#include "Fecha.h"  
  
class Empleado {  
public:  
    // Constructor: recibe nombre y dos fechas (por copia o const&)  
    Empleado(const std::string& nombre, const Fecha& fechaNac, const  
Fecha& fechaCont);  
  
    // Destructor  
    ~Empleado();  
  
    void mostrar() const;  
  
private:  
    std::string nombre;  
    Fecha fechaNacimiento;  
    Fecha fechaContratacion;  
};  
  
#endif
```

<https://www.onlinegdb.com/edit/rtDHk95fxW>

Empleado.cpp

```
#include "Empleado.h"  
#include <iostream>  
  
// Observa la lista de inicialización: primero se construyen los  
miembros (fechaNacimiento, fechaContratacion)  
// en el orden en que se declaran en la clase (no en el orden de la  
lista).  
  
Empleado::Empleado(const std::string& nom, const Fecha& nac, const  
Fecha& cont)  
    : nombre(nom), fechaNacimiento(nac), fechaContratacion(cont) {  
    std::cout << "Constructor de Empleado: " << nombre << std::endl;  
}  
  
Empleado::~~Empleado() {  
    std::cout << "Destructor de Empleado: " << nombre << std::endl;  
}
```


/**/

```
void Empleado::mostrar() const {
    std::cout << "Empleado: " << nombre << "\n";
    std::cout << "    Fecha nacimiento: ";
    fechaNacimiento.mostrar();
    std::cout << "\n    Fecha contratacion: ";
    fechaContratacion.mostrar();
    std::cout << std::endl;
}
```

<https://www.onlinegdb.com/edit/rtDHk95fxW>

main.cpp

```
#include "Empleado.h"

int main() {
    Fecha nacimiento(15, 5, 1990);
    Fecha contratacion(1, 10, 2020);

    std::cout << "\n--- Creando empleado ---\n";
    Empleado emp("Ana Garcia", nacimiento, contratacion);

    std::cout << "\n--- Mostrando datos ---\n";
    emp.mostrar();

    std::cout << "\n--- Fin del programa (se destruirá emp) ---\n";
    return 0;
}
```

<https://www.onlinegdb.com/edit/rtDHk95fxW>

salida esperada

```
Constructor de Fecha: 15/5/1990
Constructor de Fecha: 1/10/2020

--- Creando empleado ---
Constructor de Fecha: 15/5/1990    // copia del primer objeto
(fechaNacimiento)
Constructor de Fecha: 1/10/2020    // copia del segundo
(fechaContratacion)
Constructor de Empleado: Ana Garcia

--- Mostrando datos ---
Empleado: Ana Garcia
    Fecha nacimiento: 15/5/1990
    Fecha contratacion: 1/10/2020

--- Fin del programa (se destruirá emp) ---
Destructor de Empleado: Ana Garcia
Destructor de Fecha: 1/10/2020    // fechaContratacion
```

/**/

```
Destructor de Fecha: 15/5/1990    // fechaNacimiento
Destructor de Fecha: 1/10/2020    // objeto local contratacion
Destructor de Fecha: 15/5/1990    // objeto local nacimiento
```

Implementación en Python

```
class Fecha:
    def __init__(self, dia=1, mes=1, anio=2000):
        self.dia = dia
        self.mes = mes
        self.anio = anio
        print(f"Constructor de Fecha:
{self.dia}/{self.mes}/{self.anio}")

    def __del__(self):
        print(f"Destructor de Fecha:
{self.dia}/{self.mes}/{self.anio}")

    def mostrar(self):
        print(f"{self.dia}/{self.mes}/{self.anio}", end="")

class Empleado:
    def __init__(self, nombre, fecha_nac, fecha_cont):
        self.nombre = nombre
        # La composición: asignamos los objetos recibidos (ojo:
compartir referencia)
        # Para evitar compartir, podríamos copiar. Pero aquí mostramos
la composición directa.
        self.fecha_nacimiento = fecha_nac
        self.fecha_contratacion = fecha_cont
        print(f"Constructor de Empleado: {self.nombre}")

    def __del__(self):
        print(f"Destructor de Empleado: {self.nombre}")

    def mostrar(self):
        print(f"Empleado: {self.nombre}")
        print(" Fecha nacimiento: ", end="")
        self.fecha_nacimiento.mostrar()
        print()
        print(" Fecha contratacion: ", end="")
        self.fecha_contratacion.mostrar()
        print()

if __name__ == "__main__":
    nacimiento = Fecha(15, 5, 1990)
    contratacion = Fecha(1, 10, 2020)

    print("\n--- Creando empleado ---")
    emp = Empleado("Ana Garcia", nacimiento, contratacion)
```

/**/

```
print("\n--- Mostrando datos ---")
emp.mostrar()

print("\n--- Fin del programa (se destruirán objetos) ---")
# Al salir, el recolector de basura destruirá los objetos
```

<https://www.onlinegdb.com/edit/gojh6xUh6M>

SALIDA ESPERADA

```
Constructor de Fecha: 15/5/1990
Constructor de Fecha: 1/10/2020

--- Creando empleado ---
Constructor de Empleado: Ana Garcia

--- Mostrando datos ---
Empleado: Ana Garcia
  Fecha nacimiento: 15/5/1990
  Fecha contratacion: 1/10/2020

--- Fin del programa (se destruirán objetos) ---
Destructor de Fecha: 15/5/1990
Destructor de Fecha: 1/10/2020
Destructor de Empleado: Ana Garcia
```

CUADRO COMPARATIVO

Aspecto	C++	Python
Cómo se declara un objeto miembro	Fecha fechaNacimiento; (dentro de la clase Empleado)	self.fecha_nacimiento = fecha_nac (dentro de <code>__init__</code>)
Inicialización en constructor	Lista de inicialización: : fechaNacimiento(nac), fechaContratacion(cont)	Asignación directa: self.fecha_nacimiento = fecha_nac
Orden de construcción	Primero los objetos miembro (en orden de declaración), luego el objeto contenedor (Empleado)	Dentro de <code>__init__</code> , los objetos ya deben existir antes de asignarlos; no hay orden obligatorio
Orden de destrucción	Inverso a la construcción: primero el contenedor, luego sus miembros	No determinista (depende del recolector de basura), pero suele ser similar
Memoria	Los miembros son parte del objeto (se almacenan juntos)	Los miembros son referencias a objetos separados en el heap
Ejemplo simple	Empleado emp("Ana", nac, cont);	emp = Empleado("Ana", nac, cont)

/**/

Actividad 3.3 – Uso de funciones friend

Enunciado:

Declara una función friend void modificarX (Contador &, int) que modifique una variable privada x de la clase Contador.

Instrucciones:

- En C++, utiliza la palabra clave friend.
- En Python, permite acceso directo (Python no tiene friend, pero se puede simular controlando acceso).

Comparar control de acceso y seguridad.

IMPLEMENTACION EN C++

```
#include <iostream>

class Contador {
    // Declaración de función amiga (friend)
    friend void modificarX(Contador& c, int nuevoValor);

public:
    Contador() : x(0) {}

    void mostrar() const {
        std::cout << "Valor de x: " << x << std::endl;
    }

private:
    int x; // variable privada
};

// Definición de la función amiga
// Puede acceder directamente a x (privado) porque es friend
void modificarX(Contador& c, int nuevoValor) {
    c.x = nuevoValor; // Acceso directo a miembro privado
}

int main() {
    Contador cont;
    cont.mostrar(); // Valor de x: 0
    modificarX(cont, 42); // Modificación mediante friend
    cont.mostrar(); // Valor de x: 42

    // No se puede acceder a cont.x directamente desde main (error de compilación)
    // cont.x = 10; // ERROR: 'int Contador::x' is private within this context

    return 0;
}
```

<https://www.onlinegdb.com/edit/SX3DWWvXY>

```
/**/
```

SALIDA ESPERADA

```
Valor de x: 0
Valor de x: 42
```

IMPLEMENTACION EN C++

```
class Contador:
    def __init__(self):
        self.__x = 0    # Atributo "privado" (name mangling)

    def mostrar(self):
        print(f"Valor de x: {self.__x}")

# Función externa que modifica __x (simulando friend)
def modificar_x(contador, nuevo_valor):
    # Python no impide el acceso; el name mangling solo cambia el
    # nombre
    contador._Contador__x = nuevo_valor    # Acceso directo al
    "privado"

    # También se podría simplemente asignar si fuera público o
    # protegido
    # Pero respetamos la convención de que __x es privado

if __name__ == "__main__":
    c = Contador()
    c.mostrar()            # Valor de x: 0

    modificar_x(c, 99)    # Simula la función friend

    c.mostrar()            # Valor de x: 99

    # También es posible hacer esto (pero no es buena práctica)
    # c._Contador__x = 50
```

<https://www.onlinegdb.com/edit/hsRRdF8Z>

SALIDA ESPERADA

```
Valor de x: 0
Valor de x: 99
```

/**/

CUADRO COMPARATIVO

Aspecto	C++	Python
¿Existe friend?	Sí, es una palabra clave del lenguaje.	No existe.
Propósito	Permitir que una función externa acceda a miembros privados de una clase.	No se necesita; todo es público por defecto.
Cómo se declara	friend void modificarX(Contador&, int); dentro de la clase.	No hay declaración especial.
Acceso a miembros privados	La función amiga puede acceder directamente a private y protected.	Se puede acceder a cualquier atributo (incluso __x con name mangling), pero es por convención no hacerlo.
Nivel de seguridad	Controlado: solo las funciones declaradas friend tienen acceso especial.	Confianza: se espera que el programador respete las convenciones (atributo o __atributo).
Ejemplo	modificarX(cont, 42); (accede a cont.x privado)	modificar_x(cont, 42) (accede a cont._Contador__x si quiere)

Actividad 3.4 – El puntero this y llamadas encadenadas

Enunciado:

Implementa una clase Tiempo que permita llamadas encadenadas como:
t.setHora(10).setMinuto(30).setSegundo(15);

Instrucciones:

- En C++, retorna *this en cada método set.
- En Python, retorna self.

Comparar el encadenamiento de métodos.

IMPLEMENTACION EN C++

```
#include <iostream>

class Tiempo {
private:
    int hora, minuto, segundo;

public:
    Tiempo(int h = 0, int m = 0, int s = 0) : hora(h), minuto(m),
segundo(s) {}

    // Cada método set retorna una referencia al objeto actual (*this)
    Tiempo& setHora(int h) {
        if (h >= 0 && h < 24)
```

/**/

```
        hora = h;
    else
        hora = 0;
    return *this;    // Devuelve el objeto actual por referencia
}

Tiempo& setMinuto(int m) {
    if (m >= 0 && m < 60)
        minuto = m;
    else
        minuto = 0;
    return *this;
}

Tiempo& setSegundo(int s) {
    if (s >= 0 && s < 60)
        segundo = s;
    else
        segundo = 0;
    return *this;
}

void mostrar() const {
    std::cout << hora << ":" << minuto << ":" << segundo <<
std::endl;
}
};

int main() {
    Tiempo t;

    // Encadenamiento de llamadas
    t.setHora(10).setMinuto(30).setSegundo(15);
    t.mostrar();    // 10:30:15

    // También se puede encadenar la propia muestra si mostrar
    // retornara *this,
    // pero no es necesario.

    return 0;
}
```

<https://www.onlinegdb.com/edit/gvj48Amph>

SALIDA ESPERADA

10:30:15

IMPLEMENTACION EN PYTHON

```
class Tiempo:
    def __init__(self, hora=0, minuto=0, segundo=0):
        self._hora = hora
        self._minuto = minuto
```

/**/

```
self._segundo = segundo

def set_hora(self, h):
    if 0 <= h < 24:
        self._hora = h
    else:
        self._hora = 0
    return self    # Retorna self para encadenar

def set_minuto(self, m):
    if 0 <= m < 60:
        self._minuto = m
    else:
        self._minuto = 0
    return self

def set_segundo(self, s):
    if 0 <= s < 60:
        self._segundo = s
    else:
        self._segundo = 0
    return self

def mostrar(self):
    print(f"{self._hora}:{self._minuto}:{self._segundo}")

# Uso
t = Tiempo()
t.set_hora(10).set_minuto(30).set_segundo(15)
t.mostrar()    # 10:30:15

# También se puede asignar el resultado (aunque es el mismo objeto)
t2 = t.set_hora(8).set_minuto(45).set_segundo(0)
t2.mostrar()    # 8:45:0
t.mostrar()    # 8:45:0 (t y t2 son el mismo objeto)
```

<https://www.onlinegdb.com/edit/yB8FL9Lnw>

SALIDA ESPERADA

```
10:30:15
8:45:0
8:45:0
```


/**/

CUADRO COMPARATIVO

Aspecto	C++	Python
Nombre del puntero	<code>this</code> (puntero al objeto actual)	<code>self</code> (referencia al objeto actual)
Cómo se devuelve	<code>return *this;</code> (retorna una referencia)	<code>return self</code> (retorna el propio objeto)
Tipo de retorno del método	<code>Tiempo&</code> (referencia a la clase)	No se especifica tipo; Python infiere
Encadenamiento	<code>t.setHora(10).setMinuto(30).setSegundo(15);</code>	Igual: <code>t.set_hora(10).set_minuto(30).set_segundo(15)</code>
Si olvidas retornar	Error de compilación (el método retorna void por defecto)	Error en tiempo de ejecución ('NoneType' has no attribute...)
Uso típico	Configuración fluida (fluent interface)	Mismo propósito

Actividad 3.5 – Creación dinámica de objetos

Enunciado:

Crea un objeto dinámicamente de la clase Empleado usando new y elimínalo con delete.

Instrucciones:

- En C++: Utilizar punteros.
- En Python: No es necesario usar new; el recolector de basura gestiona los objetos.

Comparar gestión de memoria.

IMPLEMENTACION EN C++

```
#include "Empleado.h"
#include <iostream>

int main() {
    // Crear objetos Fecha en el stack (locales)
    Fecha nacimiento(15, 5, 1990);
    Fecha contratacion(1, 10, 2020);

    std::cout << "\n--- Creando empleado dinámicamente con new ---\n";
    // Asigna memoria en el heap para un objeto Empleado
    Empleado* empPtr = new Empleado("Carlos Lopez", nacimiento,
    contratacion);

    std::cout << "\n--- Mostrando empleado desde el puntero ---\n";
    empPtr->mostrar();
}
```

/**/

```
std::cout << "\n--- Liberando memoria con delete ---\n";
delete empPtr;    // Llama al destructor de Empleado y libera la
memoria

std::cout << "\n--- Fin del programa ---\n";
return 0;
}
```

<https://www.onlinegdb.com/edit/B9b9FnKJq>

SALIDA ESPERADA

```
Constructor de Fecha: 15/5/1990
Constructor de Fecha: 1/10/2020

--- Creando empleado dinámicamente con new ---
Constructor de Fecha: 15/5/1990    (copia para fechaNacimiento)
Constructor de Fecha: 1/10/2020    (copia para fechaContratacion)
Constructor de Empleado: Carlos Lopez

--- Mostrando empleado desde el puntero ---
Empleado: Carlos Lopez
    Fecha nacimiento: 15/5/1990
    Fecha contratacion: 1/10/2020

--- Liberando memoria con delete ---
Destructor de Empleado: Carlos Lopez
Destructor de Fecha: 1/10/2020
Destructor de Fecha: 15/5/1990

--- Fin del programa ---
Destructor de Fecha: 1/10/2020    (objeto local contratacion)
Destructor de Fecha: 15/5/1990    (objeto local nacimiento)
```

IMPLEMENTACION EN PYTHON

```
import gc

class Fecha:
    def __init__(self, dia=1, mes=1, anio=2000):
        self.dia = dia
        self.mes = mes
        self.anio = anio
        print(f"Constructor de Fecha:
{self.dia}/{self.mes}/{self.anio}")

    def __del__(self):
        print(f"Destructor de Fecha:
{self.dia}/{self.mes}/{self.anio}")

    def mostrar(self):
        print(f"{self.dia}/{self.mes}/{self.anio}", end="")
```

/**/

```
class Empleado:
    def __init__(self, nombre, fecha_nac, fecha_cont):
        self.nombre = nombre
        self.fecha_nacimiento = fecha_nac
        self.fecha_contratacion = fecha_cont
        print(f"Constructor de Empleado: {self.nombre}")

    def __del__(self):
        print(f"Destructor de Empleado: {self.nombre}")

    def mostrar(self):
        print(f"Empleado: {self.nombre}")
        print(" Fecha nacimiento: ", end="")
        self.fecha_nacimiento.mostrar()
        print()
        print(" Fecha contratacion: ", end="")
        self.fecha_contratacion.mostrar()
        print()

if __name__ == "__main__":
    nacimiento = Fecha(15, 5, 1990)
    contratacion = Fecha(1, 10, 2020)

    print("\n--- Creando empleado (sin new) ---")
    emp = Empleado("Carlos Lopez", nacimiento, contratacion)

    print("\n--- Mostrando empleado ---")
    emp.mostrar()

    print("\n--- Eliminando referencia (no obligatorio) ---")
    del emp # Reduce el contador de referencias; el GC puede liberar
    en cualquier momento

    print("\n--- Forzando recolección de basura (opcional) ---")
    gc.collect()

    print("\n--- Fin del programa ---")
```

<https://www.onlinegdb.com/edit/9RHVXee7p>

SALIDA ESPERADA

```
Constructor de Fecha: 1/10/2020

--- Creando empleado (sin new) ---
Constructor de Empleado: Carlos Lopez

--- Mostrando empleado ---
Empleado: Carlos Lopez
 Fecha nacimiento: 15/5/1990
 Fecha contratacion: 1/10/2020

--- Eliminando referencia (no obligatorio) ---
Destructor de Empleado: Carlos Lopez
```

/**/

```
--- Forzando recolección de basura (opcional) ---  
  
--- Fin del programa ---  
Destructor de Fecha: 15/5/1990  
Destructor de Fecha: 1/10/2020
```

CUADRO COMPARATIVO

Aspecto	C++	Python
Nombre del puntero	<code>this</code> (puntero al objeto actual)	<code>self</code> (referencia al objeto actual)
Cómo se devuelve	<code>return *this;</code> (retorna una referencia)	<code>return self</code> (retorna el propio objeto)
Tipo de retorno del método	<code>Tiempo&</code> (referencia a la clase)	No se especifica tipo; Python infiere
Encadenamiento	<code>t.setHora(10).setMinuto(30).setSegundo(15);</code>	Igual: <code>t.set_hora(10).set_minuto(30).set_segundo(15)</code>
Si olvidas retornar	Error de compilación (el método retorna <code>void</code> por defecto)	Error en tiempo de ejecución (<code>'NoneType' has no attribute...</code>)
Uso típico	Configuración fluida (fluent interface)	Mismo propósito